

Software Re-Engineering: Study Guide & Metrics

=== PAST PAPERS ===

Question # 3

Q: Answer the following questions considering the code of an Employee class:

```
public class Employee {
    private String name;
    private double salary;
    private int employeeId;

    public Employee (String name, double salary, int employeeId) {
        this.name = name;
        this.salary = salary;
        this.employeeId = employeeId;
    }

    public void increaseSalary(double percentage) {
        salary = salary + (salary * percentage / 100);
    }

    public String getEmployeeDetails() {
        return "ID: " + employeeId + ", Name: " + name + ", Salary: " + salary;
    }

    public boolean isSameEmployee(int id) {
        return this.employeeId == id;
    }

    public double calculateBonus(double performanceRating) {
        return salary * performanceRating * 0.1;
    }
}
```

Compute LCOM (CK), LCOM (HS) and WMC (with cyclomatic complexity) for the given class and show complete working. [10 marks]

Analyze the class for cohesion in accordance with the design principles and the metrics. [5 marks]

What conclusion can be drawn with respect to the role of metrics in evaluating design/implementation based upon the results obtained in first two parts? [5 marks]

1. Metrics Calculations

Attributes of Class (a): a = 3 (name, salary, employeeId).

Methods in Class (excluding constructor) (m): $m = 4$.

- m1 (increaseSalary): accesses {salary}
- m2 (getEmployeeDetails): accesses {name, salary, employeeId}
- m3 (isSameEmployee): accesses {employeeId}
- m4 (calculateBonus): accesses {salary}

LCOM (CK) Calculation

The CK metric counts pairs of methods that do not share attributes (P) and those that do (Q). Total method pairs: 6.

- (m1, m2) $\Rightarrow$ Share (salary)
- (m1, m3) $\Rightarrow$ No share
- (m1, m4) $\Rightarrow$ Share (salary)
- (m2, m3) $\Rightarrow$ Share (employeeId)
- (m2, m4) $\Rightarrow$ Share (salary)
- (m3, m4) $\Rightarrow$ No share

$|P| = 2$ (pairs that do not share). $|Q| = 4$ (pairs that share).

$LCOM(CK) = |P| - |Q|$ if $|P| > |Q|$, else 0. Since $2 < 4$, **LCOM (CK) = 0**.

LCOM (HS) Calculation

Excluding the constructor ($m = 4$, $a = 3$):

$\mu(\text{name}) = 1$, $\mu(\text{salary}) = 3$, $\mu(\text{employeeId}) = 2$. Sum = 6. Average = $6 / 3 = 2$.

LCOM(HS) = $(2 - 4) / (1 - 4) = -2 / -3 = 0.67$

WMC (Weighted Methods per Class)

Cyclomatic Complexity (CC) represents the number of linearly independent paths.

All methods have CC = 1 (no branches).

WMC (excluding constructor) = 4 (or 5 including constructor).

2. Cohesion Analysis

Employee exhibits low cohesion and violates the Single Responsibility Principle (SRP).

The class bundles three distinct domains: managing static entity records, payroll updates, and financial computation logic. This is captured by $LCOM(HS) = 0.67$, suggesting the class should be split into cohesive parts.

3. Role of Metrics in evaluating Design

LCOM (CK) yields 0 ("perfect cohesion"), failing to flag disjoint responsibilities. LCOM (HS) accurately identifies design degradation (0.67). Conclusion: Relying on a single binary metric like LCOM(CK) can produce false negatives. Continuous metrics must be used alongside design heuristics.

Question # 3

Q: Consider the design of a simple Digital Wallet application... (Account and Transaction classes)

Compute LCOM (HS) for both Account and Transaction classes and show complete working. [10 marks]

Identify the design defects / antipatterns in the given design with proper reasoning. [5 marks]

Suggest the design patterns to improve the design with proper reasoning. [5 marks]

1. LCOM (HS) Calculations

Account Class:

Attributes (a = 4): transactions, uuid, name, status. Methods (m = 6).

$\mu(\text{transactions}) = 3$, $\mu(\text{uuid}) = 3$, $\mu(\text{name}) = 1$, $\mu(\text{status}) = 1$.

Sum = 8. Average = 2.0.

LCOM(HS) = $(2 - 6) / (1 - 6) = -4 / -5 = 0.8$

Transaction Class:

Attributes (a = 3): amount, timestamp, memo. Methods (m = 2).

$\mu(\text{amount}) = 2$, $\mu(\text{timestamp}) = 1$, $\mu(\text{memo}) = 1$.

Sum = 4. Average = 1.33.

LCOM(HS) = $(1.33 - 2) / (1 - 2) = 0.67$

2. Design Defects / Antipatterns

Procedural Design / Data Containers: Account and Transaction function merely as passive Data Containers. The core business logic is implemented externally.

State-based conditional logic smell: External managers checking AccountStatus leads to nested conditional logic (if/else), resulting in rigid, fragile code.

3. Design Patterns to Improve the Design

State Pattern: Encapsulate account states into dedicated classes (ActiveState, SuspendedState) implementing a State interface.

Move Behavior Close to Data: Transfer operations like deposit() and withdraw() directly into Account to achieve strong data-behavior encapsulation.

Question # 3

Q: Consider the following class diagram for representation of simple HTML and XML documents...

Compute the LCOM (CK), LCOM (HS), and CBO metrics for Element class and show complete working. [10 marks]

Identify the defects in the given design and state your reasons. [5 marks]

1. Metrics Calculations

Attributes (a = 5): tagName, attributes, childElements, content, contentType. Methods (m = 5).

LCOM (CK):

Disjoint Pairs (P): (m2, m3), (m2, m4), (m3, m4). $|P| = 3$.

Sharing Pairs (Q): All others. $|Q| = 7$.

Since $|P| < |Q|$ ($3 < 7$), **LCOM (CK) = 0**.

LCOM (HS):

$\mu(\text{tagName})=1$, $\mu(\text{attributes})=3$, $\mu(\text{childElements})=3$, $\mu(\text{content})=3$, $\mu(\text{contentType})=1$.

Sum = 11. Average = 2.2.

LCOM(HS) = $(2.2 - 5) / (1 - 5) = -2.8 / -4 = 0.7$

CBO (Coupling Between Objects):

Element contains self-coupling (childElements of type Element), which is excluded.

Assuming attributes uses a custom Attribute class, **CBO = 1**. If primitive strings are used, CBO = 0.

2. Design Defects

High Cohesion Defect (High LCOM(HS) = 0.7): Aggregates two distinct conceptual structures: composite node structures and leaf node text data structures.

Cluttered/Unused State Fields: When representing a composite element, text content fields remain unused (and vice versa), leading to empty fields and memory overhead.

Question # 1

Q: Describe the role of metrics in conducting Software Re-Engineering exercises. How to make best use of them in order to improve decision making?

Role of Metrics: Provide quantitative indicators of structural software properties. They identify anomalies or "hot spots" (e.g., highly coupled, monolithic god classes) and establish baseline quality metrics before and after refactoring.

Making Best Use for Decision Making:

- Avoid Combined Formulas: Analyze individual dimensions (cohesion, coupling) simultaneously instead of using single-number indices.
- Utilize Visualizations: Multi-dimensional visuals (e.g., polymetric checker graphs) allow rapid identification of design anomalies.
- Validate Heuristics Manually: Metrics show anomalies, not necessarily structural faults. All metric-based outliers must be manually validated by browsing source code.

Question # 4(a)

Q: A developer computed CK metrics for two different classes... class A has high DIT (> 7) but low NOC (e.g. 2), class B has high NOC (> 7) but low DIT (e.g. 2). What do you deduce?

Class A (High DIT > 7, Low NOC = 2):

Deduction: Sits very deep in the inheritance hierarchy but has almost no subclasses inheriting from it.

Potential Issues: High design complexity. It inherits a large quantity of methods/attributes from its 7+ ancestors, making it difficult to comprehend, test, and debug.

Class B (High NOC > 7, Low DIT = 2):

Deduction: A base class located near the root, representing a heavily reused parent class.

Potential Issues: High impact of modification. Changes to Class B threaten all its 7+ child classes, making modifications extremely rigid and risky.

Question # 4(b)

Q: Consider the design below depicting the application of Abstract Factory pattern... Compute RFC for WinFactory class and also give your reasoning.

By definition, Response for a Class (RFC) represents the size of the response set (RS):

$$RS = \{M\} \cup (\cup \{R_i\})$$

If Constructors are Excluded:

$\{M\} = \{\text{createButton}(), \text{createText}(), \text{createContainer}()\}$ (size = 3). No other external methods are called. **RFC = 3.**

If Constructors are Included:

The factory methods invoke concrete constructors to instantiate objects. Called methods $\{R\} = \{\text{WinButton}(), \text{WinText}(), \text{WinContainer}()\}$ (size = 3). $\{M\}$ size = 4 (including WinFactory constructor).

$$\mathbf{RFC = 4 + 3 = 7.}$$

Question # 4

Q: Consider the design for a bus reservation system... Compute WMC for Bus, RFC for Reservation and Passenger, CBO for Reservation and Bus.

1. WMC for Bus Class:

WMC is the sum of method cyclomatic complexities: `updateRoute()` CC=1, `updateTimings()` CC=1, `calculateRouteDistance()` CC=6 (3 cities = 6 possible routes = 5 branches + 1), `calculateFare()` CC=2. **WMC = 10** (or 4 if not weighted).

2. RFC for Reservation Class:

Internal Methods (M = 5). Called Methods (R = 2): `amountPayable()` calls `Bus.calculateFare()` and `Bus.calculateRouteDistance()`. **RFC = 7.**

3. RFC for Passenger Class:

Internal Methods (M = 4). Called Methods (R = 2): calls `Reservation.save()` and `Reservation.amountPayable()`. **RFC = 6.**

4. CBO for Reservation Class:

Coupled to Bus (method dependency) and Passenger (association). **CBO = 2.**

5. CBO for Bus Class:

Maintains timings (`List<Time>`). **CBO = 1** (coupled to class Time).

=== BOOK QUESTIONS ===

Source: Software Evolution and Maintenance: A Practitioner's Approach, Chapter 4

Question # 11

Q: Draw a CFG for the following sample code. Determine the cyclomatic complexity of the graph.

```
(a) sum_of_all_positive_numbers(a, num_of_entries, sum)
(b) sum = 0
(c) init = 1
(d) while(init <= num_of_entries)
(e) if a[init] > 0
(f) sum = sum + a[init] endif
(g) init = init + 1 endwhile
(h) end sum_of_all_positive_numbers
```

Control Flow Graph (CFG) Structure:

Nodes (N = 6): Node 1 (Entry/Init: a,b,c), Node 2 (Loop Condition: d), Node 3 (If Condition: e), Node 4 (If Body: f), Node 5 (Increment: g), Node 6 (Exit: h).

Edges (E = 7): 1->2, 2->3, 2->6, 3->4, 3->5, 4->5, 5->2.

Cyclomatic Complexity Calculation:

$V(G) = E - N + 2P = 7 - 6 + 2(1) = 3$.

(Alternatively, $V(G) = \text{number of decision nodes} + 1 = 2 \text{ decisions (while, if)} + 1 = 3$).

Question # 1

Q: What are the differences among reengineering, reverse engineering, and forward engineering?

Reverse Engineering: Analyzing a subject system to identify its components and interrelationships to create representations at a higher level of abstraction. It strictly recovers design abstractions without modifying behavior or code.

Forward Engineering: The traditional process of moving from high-level abstractions and logical designs down to the physical implementation of a system.

Reengineering: Examination and alteration of a subject system to reconstitute it in a new form. (Reengineering = Reverse Engineering + Δ + Forward Engineering). It involves active alteration (Δ) to improve features or maintainability.

Question # 2

Q: What is software entropy and how does it relate to software aging or code decay?

Software Entropy: The natural increase in internal disorder and structural deterioration in a software system over time as it is modified. As an E-type program evolves, its complexity increases unless work is done to maintain or reduce it (Lehman's second law).

Relation to Software Aging & Code Decay: Every change introduced during software evolution (e.g., ad-hoc bug fixing) introduces structural modifications that bypass original design rules, leading to design drift. This accumulation of entropy increases resistance to future changes—a phenomenon called software aging or code decay, necessitating re-engineering.